

Lab Report 4: Introduction to Embedded Linux

System Navigation, File Permissions, and Process Control

Students: Yossi Brim & Elad Asbag

May 2026

1 Introduction

Operating systems serve as the critical interface between hardware resources and user applications. In the context of embedded electronic systems, Linux is the industry standard due to its robustness, scalability, and powerful Command Line Interface (CLI). Unlike graphical environments, the CLI allows for precise control over system resources, direct hardware interfacing, and automation via scripting.

This laboratory session focuses on the fundamental pillars of Linux administration:

1. **Filesystem Hierarchy:** Understanding the organizational structure of the OS.
2. **Access Control:** Managing security through user and group permissions.
3. **Task Management:** Controlling process execution, priority, and lifecycle.

2 Part 1: File System Navigation

2.1 Objective & Expectations

The objective was to navigate between the user's home directory and system configuration directories. **Expectation:** We expected the `cd` command to update the environment variables (verified by `pwd`) and the `ls` command to reveal the difference between user data and system-level configuration files (found in `/etc`).

2.2 Execution Log

```
localhost:~# cd /etc
localhost:/etc# pwd
/etc
localhost:/etc# ls -al | more
```

2.3 Results & Conclusion

Practical Observation: The system responded silently to successful navigation, which is standard Linux behavior. Using the `more` pager was essential in `/etc` due to the high volume of configuration files. **Conclusion:** The results perfectly aligned with expectations. We confirmed that `~root` is a shortcut to the superuser's home directory and that absolute paths (starting with `/`) allow for direct navigation regardless of the current context.

3 Part 2: File Management and Permissions

3.1 Objective & Expectations

This part focused on the Linux security model. **Expectation:** By applying the octal code `444`, we expected to strip all "Write" and "Execute" bits from the file, rendering it "Read-Only" even for the owner. We also expected the `rmdir` command to fail on non-empty directories, necessitating a recursive delete.

3.2 Execution Log

```
localhost:~# chmod 444 test.txt
localhost:~# ls -l test.txt
-r--r--r--    1 root    root                0 May 10 15:27 test.txt
localhost:~# rm -rf work
```

3.3 Results & Conclusion

Practical Observation: The `ls -l` output confirmed the permission string `-r--r--r--`. An attempt to delete a non-empty directory with `rmdir` was blocked by the kernel, while `rm -rf` bypassed this protection. **Conclusion:** The Linux permission system provides a granular method of protection. The transition from `rmdir` to `rm -rf` demonstrated the importance of recursive flags when handling complex directory trees.

4 Part 3: Process Management

4.1 Objective & Expectations

The goal was to manage asynchronous tasks. **Expectation:** We expected the `&` operator to fork a process into the background, allowing the shell to remain interactive. We also expected `pkill` to be an efficient method for terminating multiple instances of a process by name rather than individual PIDs.

4.2 Execution Log

```
localhost:~# sleep 60 &
localhost:~# jobs
[1]+  Running                  sleep 60 &
localhost:~# pkill sleep
[1]+  Terminated              sleep 60
```

4.3 Results & Conclusion

Practical Observation: The terminal successfully returned a Job ID and PID upon background execution. During the session, we observed a syntax error when placing the `&` at the beginning of a command, confirming it must be an appended operator.

Conclusion: The multitasking capabilities of Linux were verified. The ability to shift jobs between `fg` (foreground) and `bg` (background) is a vital skill for managing long-running hardware-related scripts in embedded environments.

5 Final Summary

The laboratory successfully bridged the gap between theoretical operating system concepts and practical CLI execution. Every command performed as expected according to the Linux kernel logic. These foundational skills are essential for the next stages of the course, involving Raspberry Pi hardware integration and serial communication protocols.